FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *Cette fonctionnalité permet aux administrateurs et managers de définir une vidéo d'arrière-plan immersive en tête d'un article (effet « hero »). Derrière une UI simple — drag-and-drop, preview, suppression — se cache un parcours technique contraint par les limites d'infrastructure de Netlify.*

DÉFINITION

Qu'est-ce que c'est ?

Le bloc « Vidéo Héro » est un champ optionnel sur chaque article publié dans la rubrique `savoir/`. Lorsqu'il est renseigné, la vidéo s'affiche en pleine largeur en haut de la page article, jouée en boucle silencieuse (autoplay muted loop), créant un effet cinématique propre à l'esthétique « Brume de la Rivière ».

Le champ accepte les formats `video/mp4`, `video/webm` et `video/quicktime` jusqu'à 50 MB. La vidéo est stockée dans le bucket Supabase `article-videos` et son URL publique est persistée dans la colonne `articles.hero_video_url`.

DÉCISION

Pourquoi cette implémentation, et pas une autre ?

CHOIX RETENU

Upload direct du navigateur vers Supabase Storage, sans intermédiaire serveur. L'API route Next.js initialement utilisée a été abandonnée.

Première approche (échec) : API route Next.js

La première version envoyait le fichier en `FormData` vers `/api/admin/articles/upload-hero-video`, qui validait le JWT, vérifiait le rôle, puis téléversait via le client admin Supabase (`service_role`).

POURQUOI ÇA ÉCHOUAIT

Netlify Functions imposent une limite de payload d'environ 6 MB. Toute vidéo au-delà de cette taille fait crasher la fonction **avant même** qu'elle ne s'exécute. La réponse renvoyée est un texte brut « `Internal Error. ID: 01KQ...` », ce qui faisait secondairement crasher `JSON.parse()` côté client.

Approche retenue : upload direct navigateur → Storage

Le navigateur utilise le SDK `@supabase/ssr` pour appeler directement `supabase.storage.from( ... ).upload( ... )`. Le JWT de session est attaché automatiquement par le client. Aucune fonction serveur n'est dans le chemin du fichier.

CŒUR DE L'UPLOAD CÔTÉ CLIENT TYPESCRIPT

```typescript
const { data, error } = await supabase.storage
  .from("article-videos")
  .upload(path, file, {
    cacheControl: "3600",
    upsert: false,
    contentType: file.type,
  });

if (error) { /* afficher toast */ }

const { data: urlData } = supabase.storage
  .from("article-videos")
  .getPublicUrl(data.path);

setArticle({ ...article, hero_video_url: urlData.publicUrl });
```

Pourquoi ce choix vs. les alternatives

OPTION	POUR	CONTRE	VERDICT
API route Next.js	Auth centralisée, validation côté serveur	Plafond 6 MB Netlify, latence supplémentaire	❌ Inutilisable
Signed URL (S3 presigned-style)	Pas de proxy, granularité fine	Plus de complexité (générer URL avant upload), pas natif Supabase	🟡 Possible mais surdimensionné
Upload direct + RLS	Simple, natif Supabase, pas de serveur	Sécurité dépend de RLS bien écrites	✅ Retenu
Service tiers (Cloudinary, Mux)	Optimisation vidéo automatique, CDN	Coût mensuel, dépendance externe, vendor lock-in	🟡 Trop tôt, à reconsidérer pour la Veillée Numérique

COMMENT

Architecture technique

Schéma de base de données

MIGRATION APPLIQUÉE — SQL

```sql
-- 20260426_article_hero_video.sql
ALTER TABLE public.articles
  ADD COLUMN IF NOT EXISTS hero_video_url TEXT NULL;

CREATE INDEX IF NOT EXISTS idx_articles_hero_video_url
  ON public.articles(hero_video_url);
```

Politiques RLS du bucket

Le bucket `article-videos` est public en lecture (les vidéos doivent pouvoir être servies à tout visiteur d'un article), mais écriture restreinte aux profils `admin` et `manager`.

20260427_ARTICLE_VIDEOS_STORAGE_POLICIES.SQL — SQL

```sql
CREATE POLICY "article_videos_admin_insert"
ON storage.objects
FOR INSERT
TO authenticated
WITH CHECK (
  bucket_id = 'article-videos'
  AND EXISTS (
    SELECT 1 FROM public.profiles
    WHERE profiles.id = auth.uid()
      AND profiles.role IN ('admin', 'manager')
  )
);
-- + équivalents pour UPDATE, DELETE
-- + SELECT policy publique
```

Flux complet

1. L'admin sélectionne ou drag-and-drop une vidéo dans le composant `HeroVideoCard`.
2. Validation client : MIME type ∈ {mp4, webm, quicktime}, taille ≤ 50 MB.
3. Génération d'un nom de fichier sanitisé : `articles/<id>/<timestamp>-<cleanName>.<ext>`.
4. Upload via `supabase.storage.upload()` — le SDK ajoute le JWT.
5. RLS Postgres vérifie que `auth.uid()` a le rôle requis.
6. Récupération de l'URL publique via `getPublicUrl()`.
7. Mise à jour locale du state React : `article.hero_video_url`.
8. L'admin clique sur « Publier » → la valeur est persistée en DB via `upsert`.

Indicateur de progression

Le SDK Supabase n'expose pas d'événements de progression natifs sur `upload()`. Une progression simulée est utilisée (fillrate dépendant de la taille du fichier), forcée à 100% à la résolution de la promesse. Pour une progression réelle, il faudrait passer aux uploads tus.io résumables — c'est **déjà supporté** par Supabase Storage v2 mais ajoute de la complexité non nécessaire à ce stade.

RETOUR D'EXPÉRIENCE

Difficultés rencontrées

1. JWT validation cassée (intercepté tôt)

Le code initial appelait `supabaseAdmin.auth.admin.getUserById("")` avec une chaîne vide — le JWT du client n'était jamais réellement validé. Cette erreur a été corrigée en utilisant `jwtVerify` de la librairie `jose` avec `SUPABASE_JWT_SECRET`.

2. Le package `jose` n'était pas installé

La validation JWT importait `jose` mais le package n'était pas dans `package.json`. Le build TypeScript local passait (cache de types stale) mais Netlify aurait crashé. Détecté avant déploiement grâce à `tsc --noEmit`.

3. Limite de payload Netlify (le vrai blocant)

Symptôme : **500 Internal Server Error** avec un body texte brut. Cause racine : la fonction n'arrivait jamais à s'exécuter — Netlify rejette le body avant. La solution n'était pas dans le code de la fonction mais dans l'architecture : sortir le fichier du chemin de la fonction.

> LEÇON RETENUE
>
> Pour tout fichier > 4 MB sur Netlify, ne pas le faire transiter par une Function. Soit upload direct vers le storage tiers, soit utiliser des signed URLs. Ce principe vaudra pour la future fonctionnalité **Veillée Numérique** (audio long).

4. JSON.parse sur réponse vide

Quand Netlify renvoyait son texte d'erreur, le code client tentait `response.json()` qui crashait. Le wrapper a été remplacé par `response.text()` + parsing tolérant — même si on est maintenant en upload direct, ce pattern défensif est conservé pour les autres API routes.

5. RLS manquantes (par défaut, accès refusé)

Supabase Storage applique RLS sur la table `storage.objects`. Sans politiques explicites, **aucun upload n'est autorisé**, même avec un JWT valide. Quatre politiques ont été ajoutées : INSERT/UPDATE/DELETE pour admin/manager, SELECT public.

À SAVOIR

Limitations connues

- Pas de progression réelle pendant l'upload (simulée).
- Pas de transcodage : la vidéo est servie telle quelle. Un .mov de 45 MB peut être lent à charger sur 3G.
- Pas de génération de thumbnail automatique (le poster est l'image principale de l'article).
- Pas de cleanup automatique des anciennes vidéos quand on les remplace — accumulation possible dans le bucket.

ROADMAP

Évolutions prévues

- Job nightly de garbage-collection des vidéos orphelines (non référencées par aucun article).
- Transcodage automatique vers WebM/AV1 via un service tiers (Mux ou Cloudflare Stream) si le coût se justifie.
- Génération automatique de thumbnail à la 1re seconde via FFmpeg edge function.
- Upload résumable (tus.io) pour résilience aux coupures réseau.

RÉFÉRENCE

Fichiers concernés

- `src/app/admin/content/[slug]/page.tsx` — Composant éditeur, fonction `handleVideoUpload()`
- `src/lib/admin-docs/DocLayout.tsx` — Couche de présentation
- `migrations/20260426_article_hero_video.sql` — Schéma colonne
- `migrations/20260427_article_videos_storage_policies.sql` — Politiques RLS
- `scripts/setup-storage-buckets.ts` — Création initiale du bucket

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM